SMS API Usage Guide

VIEWIN Transactional SMS Gateway — OTP Sending Reference

Credentials

Username	`karthik.trans`
Password	`_62S#uCi`
Sender ID (from)	`VIEWIN`
DLT Content ID	`1707177859908335883`
DLT Principal Entity ID	`1201159188619220464`

■ This file contains a plaintext password. Treat it as confidential — do not share it over email or public storage. Rotate the password once testing is complete, and move credentials into environment variables for any production use.

API Request Link (GET)

```
https://202.158.250.140/fe/api/v1/send?username=karthik.trans&password=_62S%23uCi&unicode=false&from=VIEWIN&to=
```

Replace **91XXXXXXXXXX** with the recipient's number (country code + 10-digit mobile) and the OTP value inside **text** before sending. The **text** must match the DLT-approved template exactly — only variable parts (like the OTP) can change.

1. curl (command line)

```
curl -k -G "https://202.158.250.140/fe/api/v1/send" \
  --data-urlencode "username=karthik.trans" \
  --data-urlencode "password=_62S#uCi" \
  --data-urlencode "unicode=false" \
  --data-urlencode "from=VIEWIN" \
  --data-urlencode "to=91XXXXXXXXX" \
  --data-urlencode "dltContentId=1707177859908335883" \
  --data-urlencode "dltPrincipalEntityId=1201159188619220464" \
  --data-urlencode "text=Welcome to View Win App! Use OTP 1234141 to complete your Registration. Valid f
```

Usage: replace 91XXXXXXXXX with the recipient number and 1234141 with the generated OTP, then run the command in a terminal.

2. Python

```
import requests
import urllib3

# Suppress SSL warning since we're using verify=False for this IP-based endpoint
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

def send_otp(to_number, otp):
    url = "https://202.158.250.140/fe/api/v1/send"

    params = {
        "username": "karthik.trans",
        "password": "_62S#uCi",
        "unicode": "false",
        "from": "VIEWIN",
        "to": f"91{to_number}",
        "dltContentId": "1707177859908335883",
        "dltPrincipalEntityId": "1201159188619220464",
        "text": f"Welcome to View Win App! Use OTP {otp} to complete your Registration. Valid for 10 min
    }

    response = requests.get(url, params=params, verify=False)
    print("Status Code:", response.status_code)
    print("Response:", response.text)
    return response.text

# Test call
send_otp("XXXXXXXXX", "1234141")  # replace XXXXXXXXX with the actual 10-digit number
```

Usage: install dependencies with `pip install requests`, then call send_otp(mobile_number, otp) with the actual 10-digit number and OTP value.

3. Node.js

```
const axios = require('axios');
const https = require('https');

const agent = new https.Agent({ rejectUnauthorized: false }); // for self-signed/IP cert

async function sendOtp(toNumber, otp) {
  const params = {
    username: 'karthik.trans',
    password: '_62S#uCi',
    unicode: 'false',
    from: 'VIEWIN',
    to: `91${toNumber}`,
    dltContentId: '1707177859908335883',
    dltPrincipalEntityId: '1201159188619220464',
    text: `Welcome to View Win App! Use OTP ${otp} to complete your Registration. Valid for 10 minutes.`
  };

  const res = await axios.get('https://202.158.250.140/fe/api/v1/send', {
    params,
    httpsAgent: agent
  });
  console.log('Response:', res.data);
  return res.data;
}

sendOtp('XXXXXXXXXX', '1234141'); // replace with actual 10-digit number
```

Usage: install dependencies with `npm install axios`, then call sendOtp(mobileNumber, otp) with the actual 10-digit number and OTP value.

General Usage Instructions

1. Generate a random OTP in your application (e.g. a 6-digit number) and store it server-side against the user's session for verification.

2. Choose any one of the three methods above (curl / Python / Node.js) based on your stack.

3. Replace the placeholder mobile number with the recipient's number in the format 91XXXXXXXXXX (country code + 10 digits, no + or spaces).

4. Insert the generated OTP into the text field — the surrounding wording must stay identical to the DLT-approved template, since DLT template mismatches cause the message to be rejected by the operator.

5. Send the request. A successful call returns a message ID / status in the response body — log this for delivery tracking.

6. On the client side, prompt the user to enter the OTP and verify it against the value stored in step 1, with an expiry (e.g. 10 minutes, matching the message text).

7. Never expose the username/password in client-side (browser/mobile app) code — always send OTPs from your backend server.